

SchedPilot

面向云原生高性能负载的 eBPF / sched_ext 用户态自适应调度系统

项目说明书

项目信息	内容
赛题	华为命题《基于 BPF/sched_ext 的用户态高性能动态调度器》
一句话定位	让内核调度器"看懂"当前跑的是什么负载，再自动把该快的调快、该让的让开
核心结果	Redis 混部场景：QPS 提升至 2.8 倍（配对 +181.5%，20/20 轮更高），p99 降三成、中位延迟降至 1/7.6
目标环境	openEuler 24.03 LTS（SP4 主验证 / SP3 兼容验证）
代码规模	eBPF 数据面 + 用户态控制面 + 实验/证据工程（C / eBPF / Shell / Python）
仓库版本	v0.3.3（HEAD 65c4485）
文档日期	2026 年 10 月

核心结果速览

一句话：在云原生混部场景下，SchedPilot 让内核调度器**"看懂"负载**——把 CPU 优先给有人在等的在线服务，把可以缓一缓的后台干扰**降权收容**（而不是饿死）。

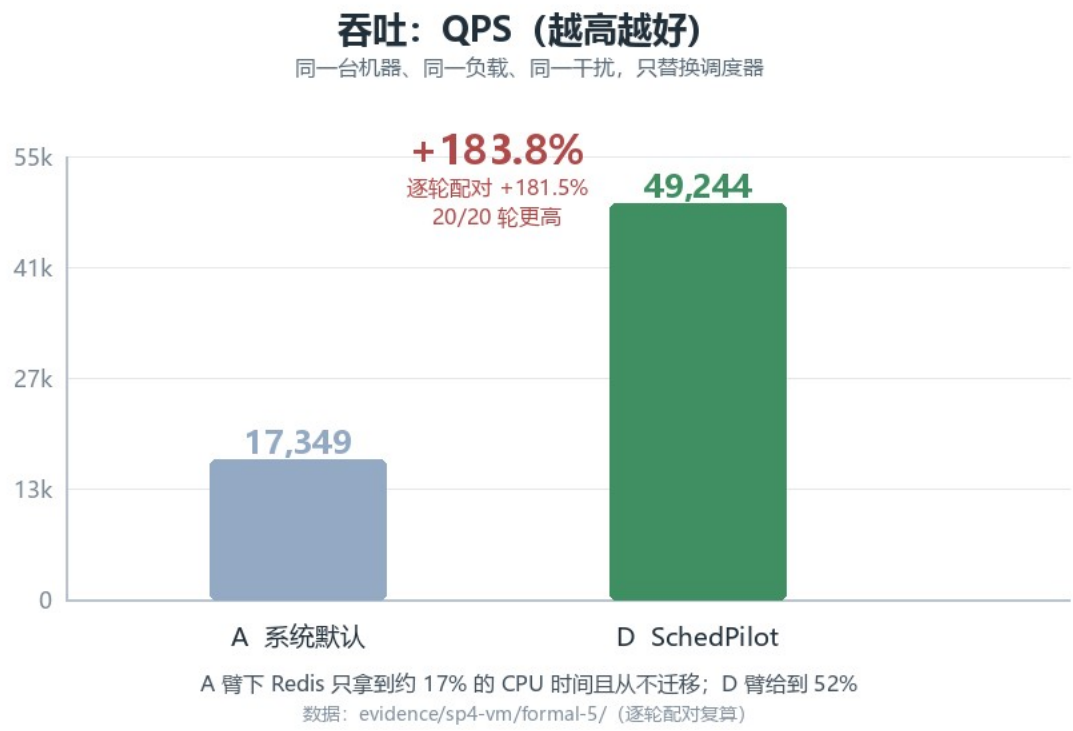


图 1 唯一变量实验：同一台机器、同一负载、同一干扰，只替换调度器（最终构建 65c4485，每臂 20 轮 × 60 秒，0 无效轮次）

看点	数字（最终构建 formal-5）
吞吐（Redis QPS 中位）	17,349 → 49,244，+183.8%；逐轮配对 +181.5%，20/20 轮更高
尾延迟 p99	4.44 ms → 2.90 ms，-34.8%，20/20 轮更低
中位延迟 p50	3.36 ms → 0.44 ms，-86.9%（约 7.6 倍）
调度器自身开销	≈0.24–0.28 μs/请求（判断每 100 ms 一次，执行全在内核）
安全与运维	loader 或判断程序被杀都不会失控；一键回滚；故障注入 6/6、15 分钟长稳 0 失速
跨版本	openEuler 24.03 LTS SP4 主验证；SP3 自编译内核零改动通过

一、项目概述

1.1 问题背景

云原生环境普遍采用"混部"来提升资源利用率：同一个 CPU 集合上既有延迟敏感的在线服务（Redis、Nginx、MySQL），又有 CPU/内存密集的批处理或干扰负载。Linux 默认的 fair-class 调度器（赛题表述为默认 CFS）对所有任务一视同仁，靠动态时间片和负载均衡维持"公平"，但它无法识别"谁是被用户等待的服务、谁是可以延后的干扰"。

结果是：在线服务的尾延迟被干扰负载拉高，同时吞吐被严重压制。本项目在演示环境中实测到一个极端但典型的现象——Redis 在混部压力下只获得 16.9% 的 CPU 时间，且调度器对它一次都不做迁移（cpu-migrations = 0）：它被"公平地"按在原地饿着。

1.2 项目目标

在 sched_ext（内核可编程调度类）之上，构建一套"感知—决策—执行—兜底"的完整闭环：

- 感知：eBPF 采集调度事件（唤醒率、运行时长、run delay、上下文切换），用户态用 PMU 采集 IPC / LLC MPKI；
- 决策：用户态每 100ms 做一次 EWMA + 非对称滞回三分类（L-SYNC / C-COMPUTE / M-BOUND，BG 显式标记）；
- 执行：内核侧按分类结果路由到 LAT / COMP / CACHE 三条 DSQ，并施加有界的切片、迁移惩罚与预抢占参数；
- 兜底：用户态进程挂掉时，数据面自动退回静态安全参数（fail-open），业务无损。

1.3 与通用方案的差异

内核态的 scx_layered 等通用方案需要人工编写分层规则，规则静态、场景绑定；本项目做的是**场景内自动分类 + 有界自适应**，并且把每一步决策都落成可审计日志。在同一混部场景下的外部对照实验（n=10）中，内核树示例调度器 scx_simple 吞吐下降 51.7%、p99 上升 351.9%；scx_flatcg 更是 10/10 轮被内核看门狗卸载，没有产生任何有效轮次。

二、系统设计

2.1 总体架构

系统由**内核数据面**与**用户态控制面**两部分组成，两者通过 pinned BPF maps 通信，构成"观测 → 判断 → 下发 → 执行 → 再观测"的闭环。

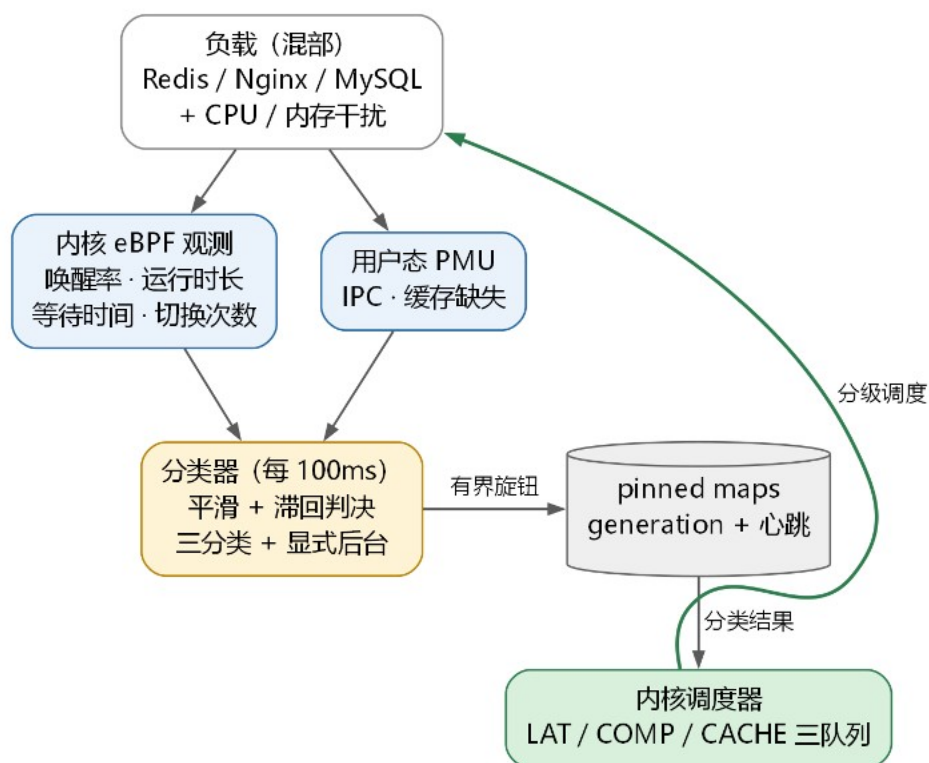


图 2 总体架构：内核负责观测与执行，用户态负责判断与下发，中间用 *pinned maps* 传递并带心跳校验

设计上最关键的一点：**判断放在用户态**（每 100 ms 判断一次，慢，但可解释、可审计），**执行放在内核里**（快，且任务不需要被转发到用户态）。

2.2 数据面（eBPF, *scx_schedpilot*）

DSQ 通道	服务对象	调度策略
LAT	L-SYNC（延迟敏感在线服务）	短切片（默认 1ms）+ 优先派发 + 唤醒预抢占
COMP	C-COMPUTE（计算型）	vtime 公平调度，避免饿死
CACHE	M-BOUND（内存型）	长切片 + LLC 软亲和 + 迁移惩罚上限
BEST	未分类任务	回退通道，保证任何任务都不会被丢弃

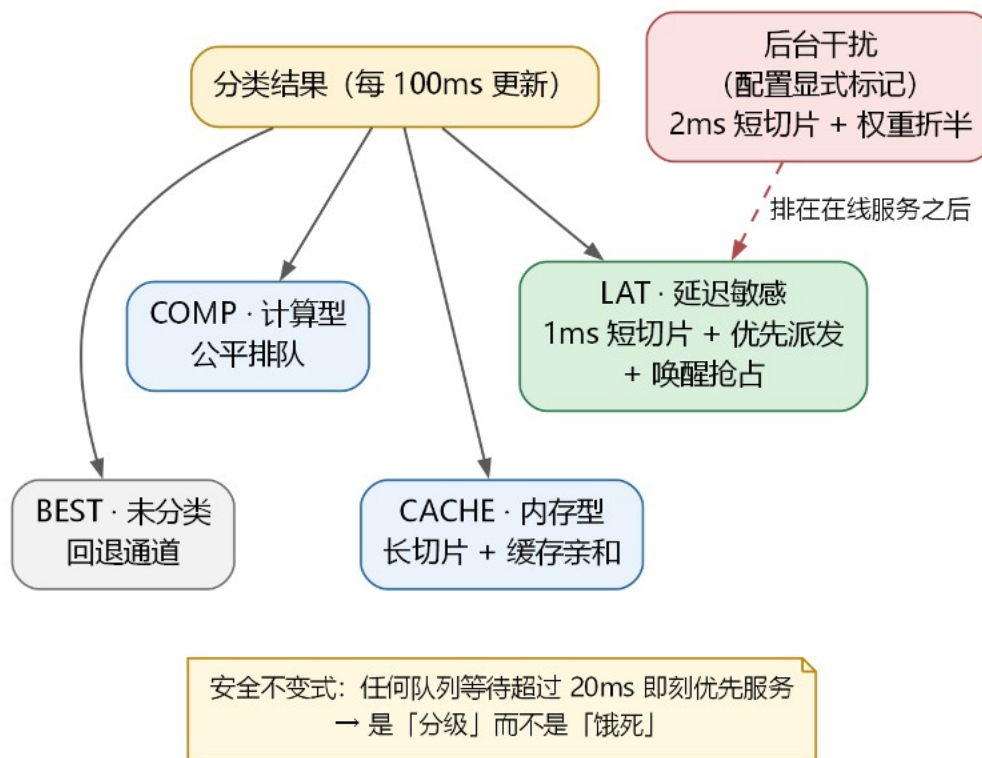


图 3 数据面：分类结果把任务路由到三条队列，未分类走回退通道；后台干扰被降权收容，而 20 ms 的抗饥饿下限保证那是“分级”而不是“饿死”

关键安全不变式：非 LAT 通道若超过 `starvation_ns`（20ms）未被派发，则被优先服务，从机制上排除“为了照顾在线服务而饿死其他任务”的风险。

2.3 控制面（用户态 `schedpilotd`）

分类器以 100ms 为决策周期（慢回路），对每个目标线程做 EWMA 平滑后按非对称滞回规则判决：

类别	判别特征（配置可调）
L-SYNC	高唤醒率（ <code>wake_hi=500</code> ）+ 适中运行时长； <code>lat_moderate</code> 规则可按负载关闭
C-COMPUTE	长运行时长 + 高 IPC（ <code>ipc_hi=1.2</code> ）
M-BOUND	高 LLC MPKI（ <code>mpki_hi=10.0</code> ）+ 长 <code>run delay</code>
BG	配置文件显式声明（如 <code>stress-ng</code> ），不做启发式猜测

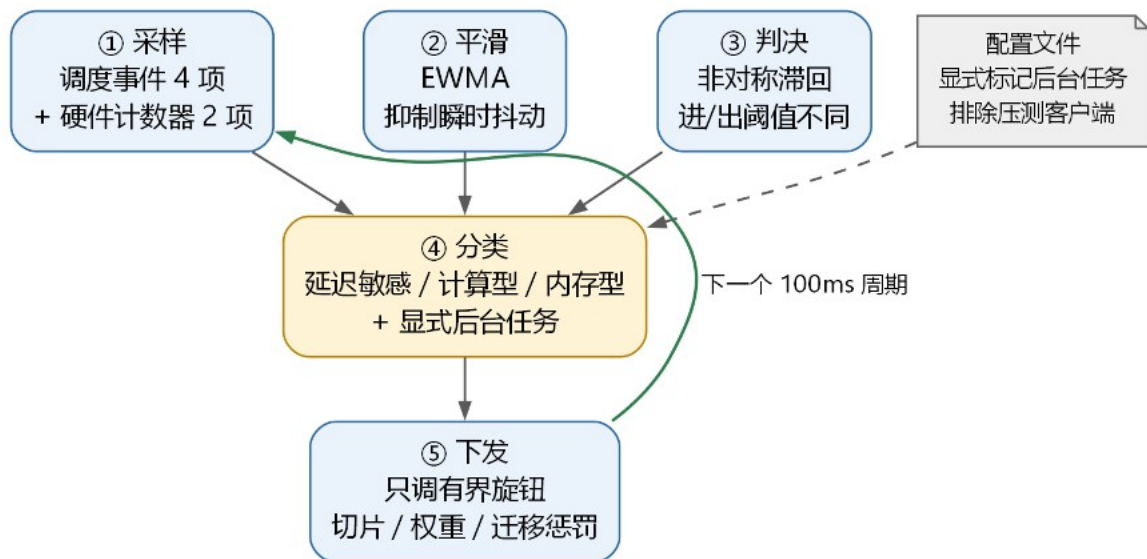


图 4 控制面：每 100 ms 一轮的决策闭环——采样、平滑、滞回判决、分类、有界下发；后台任务由配置文件显式声明，不靠启发式猜测

所有可调旋钮都带有上下界、generation 版本号与速率限制：用户态只能做"有界微调"，不能把调度器推入任意状态；这既是工程安全考虑，也让消融实验可以精确定位到单个机制。

2.4 安全与 fail-open 设计

- loader 挂掉：sched_ext 自动 detach，内核默认 fair 调度器接管，业务无损（dmesg 有 disabled 记录）；
- daemon 挂掉：BPF 数据面保持启用，心跳过期后自动退回静态安全参数，daemon 重启后 100ms 内恢复；
- 配置校验：daemon 启动时校验接口版本 intf_version，不匹配立即退出（fail-fast），杜绝旧布局静默污染；
- 一键回滚：schedpilotctl.sh rollback 停止控制面与数据面并校验 sched_ext 状态。

故障注入测试 6/6 通过，15 分钟 soak 测试 0 次失速、0 次看门狗命中。

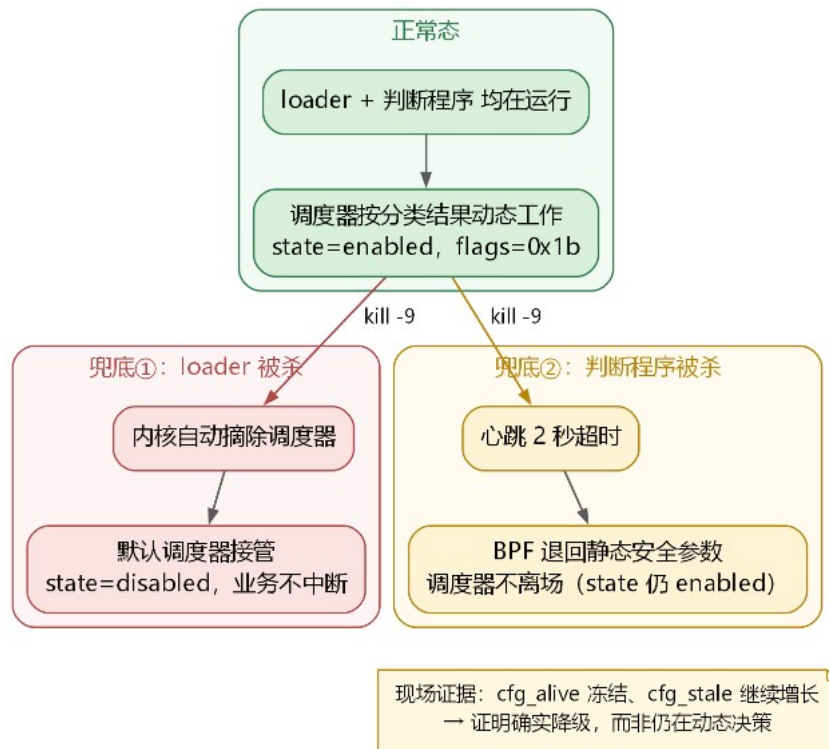


图 5 双层安全兜底：loader 被杀由内核自动摘除并交还默认调度器；判断程序被杀则调度器留在内核、退回静态安全参数（可用 `cfg_alive` / `cfg_stale` 现场证明）

三、实验设计与方法学

3.1 实验场景（场景 v3）

所有正式实验统一采用同一个可复现的混部场景，保证“唯一变量是调度器”：

项目	设置
服务 CPU 集	CPU 0-3（服务与干扰同集合，构成真实共置竞争）
客户端 CPU 集	CPU 4-7（隔离，避免压测端调度变化污染结果）
干扰负载	stress-ng: 4 个 CPU worker (matrixprod) + 2 个 VM worker (1G, vm-keep)
Redis 压测	redis-benchmark -t get -c 50, 先 10s 预热, 再按 100k 请求突发的实测速率估算总请求数
对照臂	A=默认 fair; B=basic sched_ext; C=+分类（静态策略）; D=+自适应（完整系统）
消融臂	d-no-pmu / d-no-llc / d-no-bg（逐项关闭机制）
轮次	每臂 20 轮 × 60 秒（MySQL 为 10 轮），臂顺序逐轮轮转以消除热漂移

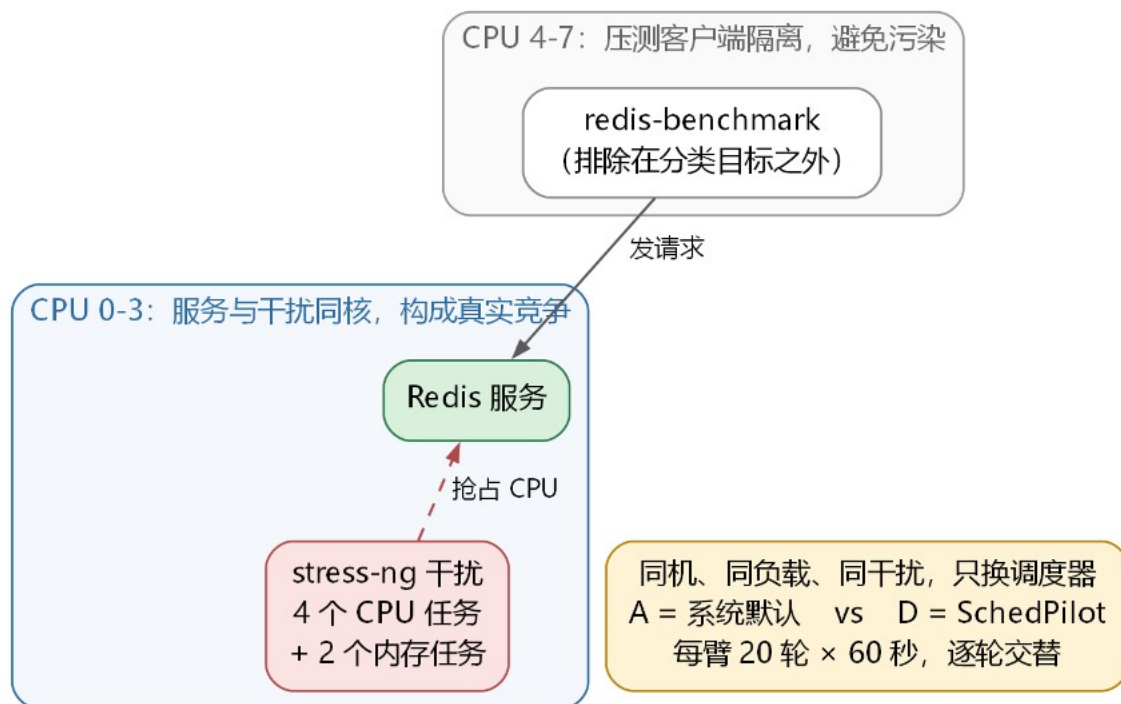


图 6 实验场景 v3: 服务与干扰共用 CPU 0-3 构成真实竞争, 压测客户端隔离在 CPU 4-7; 两臂同机同负载, 唯一变量是调度器

3.2 统计口径

我们刻意放弃了"取两个中位数比一比"的弱口径, 统一采用**逐轮配对复算**: 同一轮次内 D 与 A 直接配对求 $\Delta\%$, 再报告均值、95% 置信区间, 以及"多少轮更高/更低"。这避免了把两个独立中位数之比当成"提升幅度"的常见错误。

固定 SLO 的 **goodput** ($\text{goodput} = \text{rps} \times \text{达标请求占比}$) 作为辅助指标, 并额外报告 SLO 敏感性, 避免单一阈值带来的口径争议。

3.3 冻结与污染控制

- 实验元数据自动记录 **git commit**、三个二进制 (loader / daemon / BPF object) 的 **SHA256**、场景全部参数;
- fail-fast**: 任何一臂的装载校验失败立即中止, 绝不产出数据;
- 污染检测**: 比较测量前后的 **sched_ext enable_seq**, 若变化 (看门狗卸载或中途切换) 则整轮标记 **INVALID**;
- 负结果全部保留**: 包括一次真实回归 (formal-3, D 仅 +16.4%) 与外部对照 **scx_flatcg** 的 10/10 卸载。

四、实验结果

4.1 Redis 混部（旗舰场景：最终构建 formal-5，A/D × 20 × 60s，0 无效轮次）

指标	A（默认 fair）	D（SchedPilot）	配对提升（95% CI）	轮次优势
QPS（中位）	17349.40	49243.86	+181.49% [+174.15, +188.83]	20/20 更高
goodput@SLO(5ms)	17313.89	49240.71	+182.09% [+174.74, +189.43]	20/20 更高
p99 延迟	4.439 ms	2.899 ms	-34.80% [-35.41, -34.19]	20/20 更低
p95 延迟	4.151 ms	2.195 ms	-46.87%	20/20 更低
p50（中位）延迟	3.355 ms	0.439 ms	-86.88%（约 7.6 倍）	20/20 更低

注：本表来自**最终构建**（commit 65c4485，loader 76f0cad3 / daemon 68e57608）——与现场演示所用二进制完全相同，且已验证可从源码逐字节重建（见 evidence/sp4-vm/formal-5/rebuild-proof.txt）。前代冻结矩阵（formal-4，commit be962b8）为配对 +169.81%，两代方向与量级一致。p50/p95 由归档 per_run.csv 逐轮复算；p99 的 CI 与 p 值口径见测试报告 §6.10/§6.14。

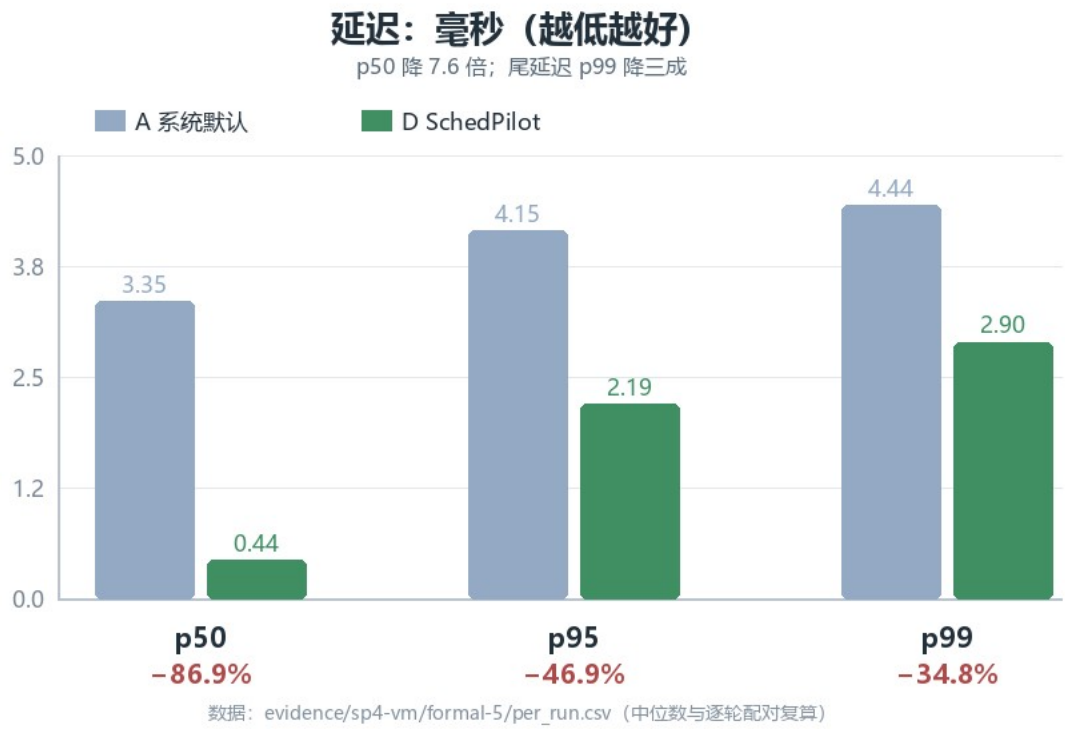


图 7 延迟对比：中位延迟降到 1/7.6，p95 降 47%，p99 降 35%（均为 20/20 轮更优）

机制层面为什么会有这种差别——perf stat 对 redis-server 进程的 60 秒窗口统计（20 轮中位数）：

臂	CPU 占用	上下文切换/s	CPU 迁移/s	IPC	LLC MPKI
A（默认 fair）	16.7%	238	0	0.694	0.93
C（分类，静态）	48.2%	1122	437	0.734	0.36
D（分类 + 自适应）	49.9%	1115	426	0.724	0.35

读法：默认 fair 下 Redis 只拿到 16.7% 的 CPU 且从不迁移；SchedPilot 让它拿到约 3 倍 CPU 时间，用更短的驻留换来 LLC MPKI 从 0.93 降到 0.35。吞吐提升主要来自"服务真的拿到了更多 CPU"，而不是统计口径上的取巧。

4.2 归因链（逐级递进）

归因链需要 B/C 臂，因此来自 7 臂冻结矩阵 formal-4（commit be962b8）；最终构建的 formal-5 只跑了 A/D 两臂以闭合溯源，未重复 B/C。

步骤	比较	QPS Δ（中位）	说明
sched_ext 本身	B vs A	+66.7%	仅把调度搬到内核可编程类，单 DSQ
任务分类	C vs B	+59.6%	三路 DSQ 路由 + BG 显式标记（静态切片）
自适应策略	D vs C	+2.5%	有界自适应旋钮的增量
总计	D vs A	+172.5%（配对 +169.8%）	—
消融 d-no-bg	vs D	-0.3%	有效对照：BG 收容位确实被清除（flags 19 vs 27）

如实说明：修复后各消融臂与 D 的差异均 ≤2.2pp 且 95% CI 相互重叠，因此我们把消融结论标注为"趋势性"，不据此宣称单一机制的贡献占比。稳健的归因是第一段（sched_ext 基础）与第二段（三分类路由），这两步的提升幅度远大于噪声。

4.3 另外两个负载

负载	实验	最佳配置	结果
MySQL	mysql-3（4 臂 × 10 × 60s）	D	TPS +85.2% [62.0, 108.3]；p99 -74.8% [-82.4, -67.3]

Nginx	nginx-7 (4 臂 × 20 × 60s)	C/D 分类模式	C +167.8%、D +169.8% (均 20/20) ; p99 均 -69.6%
-------	--------------------------	----------	---

Nginx 上自适应 (D) 与静态分类 (C) 打平 (+0.4% / -0.8%)，因此部署脚本默认使用分类模式 (adaptive)，并保留 basic 作为回退。这说明我们的自适应层不是"万能增益"——它在 Redis/MySQL 上有边际收益，在 Nginx 上保持不劣化。

4.4 无干扰回归：代价与收益的真实结构

臂	吞吐 Δ (配对)	p99 Δ (配对)	轮次
B (basic)	-5.67% [-7.15, -4.19]	-17.59%	10/10 更低
C (分类)	-3.20% [-4.00, -2.41]	-12.03%	10/10 更低
D (自适应)	-3.28% [-4.52, -2.04]	-11.04%	10/10 更低

当没有干扰时，SchedPilot 会付出约 3% 的吞吐代价，换取约 12% 的尾延迟改善。这是一个"以少量吞吐换尾延迟"的结构，我们如实保留而不加掩饰：在混部（本赛题的核心场景）下它是吞吐与延迟双赢，在无干扰下它是一种有意的取舍。

4.5 外部对照与稳定性

项目	结果
scx_simple (内核树示例)	吞吐 -51.7%，p99 +351.9% (10/10 轮更差)
scx_flatcg (内核树示例)	10/10 轮被内核看门狗卸载，无有效轮次
故障注入	6/6 PASS (loader kill → 回 fair; daemon crash → 保持 enabled 并降级; rollback 正常)
soak 长稳	15 分钟 / 49 周期, errors=0、watchdog_hits=0, final state=enabled
调度器开销	schedpilotd ≈0.24–0.28 μs/请求 (≈1.3% 单核)，0.082 次派发/请求；两个测量窗口不完全对齐，口径说明见 `docs/04_test_report.md` §6.11
SP3 兼容	自建 6.6.0-schedpilot-sp3 内核，仓库代码零改动编译通过并加载运行，迷你 A/D +129.0%

五、主要创新点

- 内核数据面 + 用户态控制面的可审计闭环：分类决策逐条落 JSONL（特征、置信度、变更原因），调度不是黑盒，现场可逐条对质；
- 有界自适应而非自由调参：所有旋钮带上下界、`generation` 版本与速率限制，用户态无法把内核调度器推入未定义状态；
- 显式 BG 标记 + 收容机制：干扰任务由配置显式声明（不靠启发式猜测），用短切片 + `vtime` 惩罚收容，而不是简单地把它们饿死；
- 双层 fail-open：loader 与 daemon 分别失效都有明确的降级路径，并且可以用 `flags`（27 → 15）现场验证降级确实发生；
- 把"负结果"当一等信息：真实回归（`formal-3`）、被看门狗卸载的外部调度器、无干扰下的吞吐代价，全部保留在证据链中。

六、工程规范与可复现性

- 证据链：每次实验写入 `experiment.meta.json`（commit + 三个二进制 SHA256 + 完整场景参数），原始数据与归档 `tar` 均带 `sha256`；
- 自动化：`scripts/`（环境自检、构建、生命周期控制、一键演示、状态页）、`bench/`（实验驱动、干扰控制、三种负载的解析器、统计分析）、`tests/`（分类器单测、故障注入、`soak`）；
- CI：GitHub Actions 覆盖 `shell/python` 检查、daemon 构建、分类器单元测试与 BPF 构建路径；
- 一键演示：`scripts/demo.sh` 完成自检 → 干扰 → A/D 两轮 → 汇总 → 回滚（实测约 97 秒）。

七、已知局限与后续工作

我们主动列出以下边界，而不是回避：

局限	说明与后续计划
冻结矩阵与最终构建的版本差	正式冻结统计（+169.8%）来自 <code>v0.3.1</code> 构建（commit <code>be962b8</code> ，二进制 <code>9d1e904f/d02d4e0d</code> ）。我们在最终构建（HEAD <code>65c4485</code> ，二进制 <code>76f0cad3/68e57608</code> ）上做了两件事：① 干净重建，产物与现场二进制逐字节一致，证明构建可复现、现场二进制即该源码树的产物；② 用现场这套二进制补跑正式矩阵 <code>formal-5</code> （A/D × 20 轮 × 60s，0 无效轮次），得到 QPS 中位 +183.8%、配对 +181.5%（95% CI [+174.2, +188.8]，20/20 轮更高）、p99 -34.8%、p50 -86.9%；A 基线几乎不变

	（17274→17349），D 臂提升约 4.6%。归档见 evidence/sp4-vm/formal-5/ 。
LLC 消融在本演示 VM 上不构成有效对照	演示机是 KVM 客户机，每个 vCPU 独立暴露一个 cache 域（16 domains / 16 cpus）。daemon 按设计自动关闭 waker-LLC 路由并记录 WARN，因此 d-no-llc 臂的 cfg flags 与 D 臂完全相同（均为 27）。LLC 软亲和能力需在具备真实 cache 拓扑的物理机上验证；当前有效的机制消融是 d-no-bg（flags 19 vs 27）。
固定 SLO 指标在 5ms 阈值下退化为 QPS	基线 p99 为 4.45ms，低于 5ms 的 SLO，阈值不"咬"，goodput 与 QPS 几乎相等。我们改用 SLO 敏感性分析：4.0ms 时 A 达标率降至 87.6%、D 保持 99.99%，goodput 配对提升 +207.7%。该重算基于归档原始数据，无需重跑。
NUMA 为观测而非决策	当前仅输出 numa_local_pct / numa_nodes（report-only），尚未参与 CACHE 路由决策；且演示机为单 NUMA 节点，该字段无信息量。列为下一步工作。
BPF 回调开销未精确折算	本 6.6 backport 内核不为 struct_ops 程序暴露 run_time_ns，无法精确测量内核侧回调时间；当前以派发频次（0.082 次/请求）+ 0 watchdog + 1Hz 策略周期作上界定性保证。
容器资源模型	已支持 cgroup v2 子树目标选择（递归读取 cgroup.procs + 白名单校验，回归测试 5/5 PASS），结合容器 QoS/配额做类优先级映射列为 P1。
BG 收容的代价未直接测量	BG 收容机制是"有界降权"而非"饿死"：vtime 惩罚 ×2、切片收敛到 2ms，并在非 LAT 通道设置 20ms 抗饥饿上限。但实验只记录了服务侧的 CPU/延迟收益，未记录干扰任务自身的完成量或进度。补测干扰侧吞吐是下一步最该做的一项。
Nginx/MySQL 的基线存在双峰波动	Redis 基线很稳（QPS IQR 仅 494），但 Nginx 与 MySQL 的 A 臂单轮值呈双峰（IQR 约为中位数的 35%），而 B/C/D 各臂 IQR 仅 1%–3%。这会让"基线抖动"看起来大于效应；我们依靠逐轮配对统计处理（配对后再比较，17–20/20 轮同向），但该事实此前未在文档中披露。
指标口径未完全固化	固定 SLO 的阈值是环境变量（默认 5.0 ms）且未写入实验元数据；部分早期矩阵的配对统计由原始输出事后复算，其 summary.md 不含该表。后续会把 SLO 与统计口径一并写入 experiment.meta.json。

附录：复现指引

环境要求：openEuler 24.03 LTS（SP4 主 / SP3 兼容），内核需启用

CONFIG_SCHED_CLASS_EXT（可在发行内核源码树上仅新增该选项并重建，本项目在 SP3/SP4 上均验证通过）。

关键命令：

```
scripts/env_check.sh --json evidence/env_check.json      # 能力探测（sched_ext / BTF /  
PMU / 工具链）  
  
scripts/build.sh --kernel-src <内核源码树> --install    # 构建数据面 + 控制面  
  
scripts/schedpilotctl.sh start --mode adaptive --config configs/redis.conf  
  
scripts/schedpilotctl.sh status / rollback  
  
scripts/demo.sh 30 1                                     # 一键 A vs D 演示（约 97 秒）  
  
bench/abcd_experiment.sh --workload redis --runs 20 --duration 60 --warmup 10 --arms  
A,B,C,D,d-no-pmu,d-no-llc,d-no-bg
```

详细结果与原始证据：docs/04_test_report.md、submission/evidence_index.md、evidence/。

现场演示流程：docs/05_demo_runbook.md（实机校准版）；答辩风险与

Q&A：docs/06_demo_risks_qa.md。